

Chapter 5 — Prefix Sum

0. What You Will Learn in This Chapter

Prefix Sum is one of the most important patterns for array, string, and matrix problems.

It is used when you need to answer questions about:

- range sums
- subarray sums
- cumulative totals
- left sum vs right sum
- pivot/equilibrium index
- count of subarrays with sum K
- longest subarray with sum K
- subarrays divisible by K
- binary array balance problems
- 2D matrix range sums
- prefix/suffix product-style thinking

By the end of this chapter, you should understand:

- What prefix sum means
- Why prefix sum avoids repeated summation
- Inclusive vs exclusive prefix arrays
- How to answer range sum queries in $O(1)$
- How prefix sum transforms subarray problems
- How prefix sum combines with hash maps
- When prefix sum is better than sliding window
- How to handle negative numbers
- How to handle modulo prefix sums
- How to build 2D prefix sums
- Common off-by-one mistakes
- Common interview explanations

Core LeetCode problems covered in this chapter:

- 1.1. Running Sum of 1D Array
 - 2.2. Range Sum Query — Immutable
 - 3.3. Find Pivot Index
 - 4.4. Find the Highest Altitude
 - 5.5. Product of Array Except Self
 - 6.6. Subarray Sum Equals K
 - 7.7. Contiguous Array
 - 8.8. Subarray Sums Divisible by K
 - 9.9. Continuous Subarray Sum
 - 10.10. Binary Subarrays With Sum
 - 11.11. Count Number of Nice Subarrays
 - 12.12. Range Sum Query 2D — Immutable
 - 13.13. Matrix Block Sum
 - 14.14. Maximum Size Subarray Sum Equals K
 - 15.15. Number of Submatrices That Sum to Target
-

1. What Is Prefix Sum?

A prefix sum is a running cumulative sum.

Given:

nums = {2, 4, 1, 3}

The prefix sums are:

prefix = {2, 6, 7, 10}

Meaning:

prefix[0] = nums[0]

prefix[1] = nums[0] + nums[1]

prefix[2] = nums[0] + nums[1] + nums[2]

prefix[3] = nums[0] + nums[1] + nums[2] + nums[3]

So prefix sum stores:

Sum of everything before or up to a position.

2. Why Prefix Sum Exists

Suppose you need to answer many range sum queries.

Example:

nums = {2, 4, 1, 3, 6, 5}

Query:

sum from index 1 to index 4

That sum is:

$4 + 1 + 3 + 6 = 14$

Without prefix sum, you loop from 1 to 4.

If there are many queries, this becomes slow.

Prefix sum lets you answer each range sum in $O(1)$ after $O(n)$ preprocessing.

3. Core Prefix Sum Formula

There are two common ways to build prefix sum.

3.1 Inclusive Prefix Sum

Inclusive prefix means:

prefix[i] = nums[0] + nums[1] + ... + nums[i]

Example:

nums = {2, 4, 1, 3}

prefix = {2, 6, 7, 10}

Range sum from l to r:

$\text{sum}(l, r) = \text{prefix}[r] - \text{prefix}[l - 1]$

But if $l == 0$, then:

```
sum(0, r) = prefix[r]
```

So inclusive prefix needs a special case.

3.2 Exclusive Prefix Sum With Leading Zero

This is the recommended version.

Define:

```
prefix[0] = 0  
prefix[i + 1] = prefix[i] + nums[i]
```

Example:

```
nums = {2, 4, 1, 3}  
prefix = {0, 2, 6, 7, 10}
```

Now:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

No special case is needed.

Example:

Sum from index 1 to 3:

```
nums[1] + nums[2] + nums[3] = 4 + 1 + 3 = 8
```

Formula:

```
prefix[4] - prefix[1] = 10 - 2 = 8
```

4. Core Need / Why Use Prefix Sum Over Other Patterns?

Prefix Sum is useful when repeated range calculations can be converted into subtraction.

4.1 Why Not Basic Traversal?

If you only need one total sum, basic traversal is enough.

But if you need many range sums, basic traversal repeats work.

Example:

```
sumRange(0, 3)  
sumRange(2, 5)  
sumRange(1, 4)  
sumRange(0, 5)
```

Without prefix sum:

Each query scans a range.

With prefix sum:

Each query is $O(1)$.

4.2 Why Not Sliding Window?

Sliding window is excellent when:

- numbers are positive
- you need longest/shortest window
- window validity changes monotonically

But prefix sum is better when:

- negative numbers exist
- exact sum K is needed
- count of subarrays is needed
- range queries are independent
- modulo relationships are involved

Example:

nums = {1, -1, 1}

k = 1

Sliding window struggles because adding/removing numbers does not behave predictably.

Prefix sum handles it cleanly.

4.3 Why Hash Map Is Often Combined With Prefix Sum

Prefix sum transforms subarray sum into a relationship between two prefix sums.

For subarray sum K:

currentPrefix - previousPrefix = k

Rearrange:

previousPrefix = currentPrefix - k

So now the problem becomes:

Have I seen currentPrefix - k before?

That is a hash map lookup.

This is why many advanced prefix sum problems are really:

Prefix Sum + Hash Map

5. When to Use Prefix Sum

Use Prefix Sum when you see these clues.

5.1 Range Sum Queries

Problem says:

- sum from left to right
- multiple queries
- immutable array
- matrix sum region

Examples:

- Range Sum Query — Immutable
- Range Sum Query 2D — Immutable
- Matrix Block Sum

5.2 Subarray Sum

Problem says:

- subarray sum equals K
- count subarrays with sum K
- longest subarray with sum K
- sum divisible by K
- continuous subarray sum

Examples:

- Subarray Sum Equals K
 - Maximum Size Subarray Sum Equals K
 - Subarray Sums Divisible by K
 - Continuous Subarray Sum
-

5.3 Left Sum vs Right Sum

Problem says:

- pivot index
- equilibrium index
- left sum equals right sum
- middle index

Examples:

- Find Pivot Index
 - Find the Middle Index in Array
 - Left and Right Sum Differences
-

5.4 Cumulative Change

Problem says:

- running sum
- gain
- altitude
- balance over time
- net change

Examples:

- Running Sum of 1D Array
 - Find the Highest Altitude
-

5.5 Modulo / Divisibility

Problem says:

- divisible by K
- multiple of K
- same remainder
- modulo

Examples:

- Subarray Sums Divisible by K
 - Continuous Subarray Sum
 - Make Sum Divisible by P
-

5.6 Binary Balance Transformations

Problem says:

- equal number of 0 and 1
- equal counts
- balanced binary array

Common trick:

0 -> -1

1 -> +1

Then equal 0s and 1s means subarray sum is 0.

Example:

- Contiguous Array

6. When NOT to Use Prefix Sum

6.1 Range Updates

If the problem gives many range updates like:

add value to every element from l to r

Use Difference Array.

Prefix sum is for range queries.

Difference array is for range updates.

6.2 Dynamic Updates + Range Queries

If values change and you must answer range sums after updates, use:

- Fenwick Tree
- Segment Tree

Example:

- Range Sum Query — Mutable

6.3 Range Minimum / Maximum

Prefix sum only helps with sums.

For min/max over range, use:

- segment tree
- sparse table
- monotonic deque
- heap in some cases

6.4 Non-Contiguous Subsequences

Prefix sum is mainly for contiguous ranges.

For subsequences, use:

- DP
- greedy
- backtracking

7. Mental Models

7.1 Mile Marker Model

Imagine array elements as distances between mile markers.

Prefix sum tells your position at each marker.

Distance from marker l to marker $r + 1$ is:

$$\text{prefix}[r + 1] - \text{prefix}[l]$$

7.2 Bank Balance Model

Prefix sum is like your bank balance over time.

If balance now is 100 and balance earlier was 70, then change between then and now is:

$$100 - 70 = 30$$

Similarly:

$$\text{subarray sum} = \text{currentPrefix} - \text{previousPrefix}$$

7.3 Subarray as Difference of Prefixes

Any subarray sum can be represented as:

$$\text{prefix after right} - \text{prefix before left}$$

For subarray $\text{nums}[l..r]$:

$$\text{sum} = \text{prefix}[r + 1] - \text{prefix}[l]$$

This is the core idea of the chapter.

7.4 Repeated Prefix Means Zero Sum

If the same prefix sum appears twice:

$$\text{prefix}[i] == \text{prefix}[j]$$

Then the sum between them is zero.

Why?

$$\text{prefix}[j] - \text{prefix}[i] = 0$$

This is useful for:

- Contiguous Array
- longest zero-sum subarray
- repeated balance states

7.5 Same Remainder Means Divisible Difference

If two prefix sums have the same remainder modulo K :

$$\text{prefixA} \% k == \text{prefixB} \% k$$

Then their difference is divisible by K .

This is the key for:

- Subarray Sums Divisible by K
- Continuous Subarray Sum

8. Pattern Recognition Checklist

Ask:

2. Do I need many range sums?
3. Is the problem about subarray sums?
4. Does it ask for exact sum K?
5. Does the array contain negative numbers?
6. Does sliding window fail because negatives exist?
7. Do I need count of subarrays?
8. Do I need longest subarray with a given sum?
9. Is divisibility/modulo involved?
10. Do I need left sum and right sum?
11. Is this a matrix rectangle sum problem?

If yes, Prefix Sum is likely useful.

9. C++ Tools Needed

9.1 `vector<int> prefix`

```
vector<int> prefix(n + 1, 0);
```

9.2 `long long`

Use if sums can be large.

```
long long prefix = 0;  
vector<long long> prefix(n + 1, 0);
```

9.3 `unordered_map`

Used for prefix sum counts or first indices.

```
unordered_map<int, int> count;  
unordered_map<int, int> firstIndex;
```

For large sums:

```
unordered_map<long long, int> count;
```

9.4 Modulo Normalization

In C++, negative modulo can be negative.

Example:

```
-1 % 5 == -1
```

To normalize:

```
int rem = ((prefix % k) + k) % k;
```

This ensures rem is between 0 and k - 1.

10. Core Templates

Template 1 — Exclusive Prefix Array

Recommended for range sums.

```
vector<int> prefix(nums.size() + 1, 0);

for (int i = 0; i < nums.size(); i++) {
    prefix[i + 1] = prefix[i] + nums[i];
}
```

Range sum from l to r:

```
int sum = prefix[r + 1] - prefix[l];
```

Template 2 — Inclusive Prefix Array

```
vector<int> prefix(nums.size());

prefix[0] = nums[0];

for (int i = 1; i < nums.size(); i++) {
    prefix[i] = prefix[i - 1] + nums[i];
}
```

Range sum:

```
int sum;

if (l == 0) {
    sum = prefix[r];
} else {
    sum = prefix[r] - prefix[l - 1];
}
```

Works, but has special cases.

Template 3 — Running Prefix Without Array

Use when you only need current cumulative sum.

```
int prefix = 0;

for (int x : nums) {
    prefix += x;

    // use prefix
}
```

Template 4 — Prefix Sum + Hash Map Count

Used to count subarrays with sum K.

```
unordered_map<int, int> count;
count[0] = 1;

int prefix = 0;
int answer = 0;

for (int x : nums) {
    prefix += x;

    int need = prefix - k;

    if (count.count(need)) {
        answer += count[need];
    }

    count[prefix]++;
}

return answer;
```

Template 5 — Prefix Sum + First Index

Used for longest subarray with sum K.

```
unordered_map<int, int> firstIndex;
firstIndex[0] = -1;

int prefix = 0;
int best = 0;

for (int i = 0; i < nums.size(); i++) {
    prefix += nums[i];

    int need = prefix - k;

    if (firstIndex.count(need)) {
        best = max(best, i - firstIndex[need]);
    }

    if (!firstIndex.count(prefix)) {
```

```
        firstIndex[prefix] = i;
    }
}
```

return best;

Important:

Store the first occurrence only.

Why?

For longest length, earliest prefix gives largest distance.

Template 6 — Prefix Modulo Count

Used for counting subarrays divisible by K.

```
unordered_map<int, int> count;
count[0] = 1;
```

```
int prefix = 0;
int answer = 0;
```

```
for (int x : nums) {
    prefix += x;
```

```
    int rem = ((prefix % k) + k) % k;
```

```
    if (count.count(rem)) {
        answer += count[rem];
    }
```

```
    count[rem]++;
}
```

```
return answer;
```

Template 7 — Prefix Modulo First Index

Used to check existence of length at least 2 subarray whose sum is a multiple of K.

```
unordered_map<int, int> firstIndex;
firstIndex[0] = -1;
```

```
int prefix = 0;
```

```
for (int i = 0; i < nums.size(); i++) {
    prefix += nums[i];
```

```

int rem = prefix % k;

if (firstIndex.count(rem)) {
    if (i - firstIndex[rem] >= 2) {
        return true;
    }
} else {
    firstIndex[rem] = i;
}
}

return false;

```

Template 8 — 2D Prefix Sum

```

int rows = matrix.size();
int cols = matrix[0].size();

vector<vector<int>> prefix(rows + 1, vector<int>(cols + 1, 0));

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        prefix[r + 1][c + 1] =
            matrix[r][c]
            + prefix[r][c + 1]
            + prefix[r + 1][c]
            - prefix[r][c];
    }
}

```

Query sum from (r1, c1) to (r2, c2):

```

int sum =
    prefix[r2 + 1][c2 + 1]
    - prefix[r1][c2 + 1]
    - prefix[r2 + 1][c1]
    + prefix[r1][c1];

```

11. Major Variations of Prefix Sum

Running prefix	cumulative total	Running Sum

Variation	Core Idea	Example
Range query	$\text{prefix}[r+1] - \text{prefix}[l]$	Range Sum Query
Left/right sum	total - left - current	Pivot Index
Prefix + count map	count previous prefixes	Subarray Sum Equals K
Prefix + first index	longest distance	Maximum Size Subarray Sum Equals K
Prefix modulo	same remainder	Subarray Sums Divisible by K
Binary balance	0 -> -1, 1 -> +1	Contiguous Array
Prefix/suffix	left info + right info	Product Except Self
2D prefix	rectangle sum	Range Sum Query 2D

12. Full Problem Walkthrough 1 — LeetCode 1480: Running Sum of 1D Array

12.1 Problem Summary

Given array `nums`, return running sum array.

Example:

`nums = {1, 2, 3, 4}`

Output:

`{1, 3, 6, 10}`

12.2 How to Identify This Pattern

Problem directly says:

- running sum
- cumulative sum

This is the simplest prefix sum problem.

12.3 Brute Force Idea

For each index i , sum from 0 to i .

Time: $O(n^2)$

12.4 Optimized Intuition

Keep a running sum:

```
sum += nums[i];
```

12.5 C++ Code

```
class Solution {
public:
    vector<int> runningSum(vector<int>& nums) {
        vector<int> result(nums.size());

        int sum = 0;

        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i];
            result[i] = sum;
        }

        return result;
    }
};
```

12.6 Dry Run

nums = {1, 2, 3, 4}

0	1	1	{1, _, _ }
1	2	3	{1, 3, _ }
2	3	6	{1, 3, 6, _ }
3	4	10	{1, 3, 6, 10}

12.7 Complexity

Time: $O(n)$

Space: $O(n)$, because result array is returned.

12.8 Similar Problems

- Find the Highest Altitude
- Range Sum Query — Immutable
- Find Pivot Index

13. Full Problem Walkthrough 2 — LeetCode 303: Range Sum Query — Immutable

13.1 Problem Summary

Design a class that supports range sum queries on an immutable array. Implement:

```
NumArray(vector<int>& nums)
int sumRange(int left, int right)
```

Example:

```
nums = {-2, 0, 3, -5, 2, -1}
sumRange(0, 2) = 1
sumRange(2, 5) = -1
sumRange(0, 5) = -3
```

13.2 How to Identify This Pattern

Problem asks:

- multiple range sum queries
- array is immutable
- need efficient query

Use prefix sum.

13.3 Brute Force Idea

Every query loops from left to right.
If many queries exist, this is slow.

13.4 Optimized Intuition

Precompute exclusive prefix array:

```
prefix[0] = 0
prefix[i + 1] = prefix[i] + nums[i]
```

Then:

```
sumRange(left, right) = prefix[right + 1] - prefix[left]
```

13.5 C++ Code

```
class NumArray {
private:
    vector<int> prefix;

public:
```

```

NumArray(vector<int>& nums) {
    prefix.resize(nums.size() + 1, 0);

    for (int i = 0; i < nums.size(); i++) {
        prefix[i + 1] = prefix[i] + nums[i];
    }
}

int sumRange(int left, int right) {
    return prefix[right + 1] - prefix[left];
}
};

```

13.6 Code Explanation

`vector<int> prefix;`

Stored as member variable so all methods can access it.

`prefix.resize(nums.size() + 1, 0);`

Extra leading zero avoids special cases.

`return prefix[right + 1] - prefix[left];`

Range sum formula.

13.7 Dry Run

`nums = {-2, 0, 3, -5, 2, -1}`

Prefix:

`prefix = {0, -2, -2, 1, -4, -2, -3}`

Query:

`sumRange(2, 5)`

Formula:

`prefix[6] - prefix[2] = -3 - (-2) = -1`

13.8 Complexity

Constructor:

Time: $O(n)$

Space: $O(n)$

Each query:

Time: $O(1)$

13.9 Edge Cases

- query starts at index 0

- query is one element
 - negative numbers
 - whole array query
-

13.10 Common Mistakes

Wrong formula for exclusive prefix:

$\text{prefix}[\text{right}] - \text{prefix}[\text{left}]$

Correct:

$\text{prefix}[\text{right} + 1] - \text{prefix}[\text{left}]$

13.11 Similar Problems

- Range Sum Query 2D — Immutable
 - Range Sum Query — Mutable, Fenwick/Segment Tree
 - Matrix Block Sum
-

14. Full Problem Walkthrough 3 — LeetCode 724: Find Pivot Index

14.1 Problem Summary

Given array `nums`, return the leftmost index where:

sum of elements left of index == sum of elements right of index

Example:

`nums = {1, 7, 3, 6, 5, 6}`

Output:

3

Left sum:

$1 + 7 + 3 = 11$

Right sum:

$5 + 6 = 11$

14.2 How to Identify This Pattern

Problem asks:

- left sum
- right sum
- equilibrium/pivot index

Use total sum and running left sum.

14.3 Brute Force Idea

For each index:

- compute sum left
- compute sum right

Time: $O(n^2)$

14.4 Optimized Intuition

Compute total once.

While scanning:

```
rightSum = total - leftSum - nums[i]
```

If:

```
leftSum == rightSum
```

return index.

14.5 C++ Code

```
class Solution {
public:
    int pivotIndex(vector<int>& nums) {
        int total = 0;

        for (int x : nums) {
            total += x;
        }

        int leftSum = 0;

        for (int i = 0; i < nums.size(); i++) {
            int rightSum = total - leftSum - nums[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }
};
```

14.6 Dry Run

```
nums = {1, 7, 3, 6, 5, 6}
```

Total:

28

0	1	0	27	no
1	7	1	20	no
2	3	8	17	no
3	6	11	11	yes

Return 3.

14.7 Complexity

Time: $O(n)$

Space: $O(1)$

14.8 Edge Cases

- pivot at index 0
- pivot at last index
- negative numbers
- no pivot
- multiple pivots, return leftmost

14.9 Common Mistake

Updating left sum before checking pivot.

Correct order:

check pivot
then `leftSum += nums[i]`

14.10 Similar Problems

- Find the Middle Index in Array
- Left and Right Sum Differences
- Equilibrium index problems

15. Full Problem Walkthrough 4 — LeetCode 1732: Find the Highest Altitude

15.1 Problem Summary

You start at altitude 0.

Given gain, where `gain[i]` is altitude change, return highest altitude reached.

Example:

gain = {-5, 1, 5, 0, -7}

Altitudes:

0, -5, -4, 1, 1, -6

Output:

1

15.2 How to Identify This Pattern

Problem says:

- start at 0
- gain/change
- highest cumulative value

Use running prefix sum + max.

15.3 C++ Code

```
class Solution {
public:
    int largestAltitude(vector<int>& gain) {
        int altitude = 0;
        int best = 0;

        for (int x : gain) {
            altitude += x;
            best = max(best, altitude);
        }

        return best;
    }
};
```

15.4 Dry Run

gain = {-5, 1, 5, 0, -7}

start	0	0
-5	-5	0
1	-4	0
5	1	1
0	1	1

gain	altitude	best
-7	-6	1

15.5 Complexity

Time: $O(n)$

Space: $O(1)$

15.6 Similar Problems

- Running Sum of 1D Array
 - Pivot Index
 - Prefix balance problems
-

16. Full Problem Walkthrough 5 — LeetCode 238: Product of Array Except Self

16.1 Problem Summary

Given array `nums`, return array `answer` where:

`answer[i] = product of all nums except nums[i]`

You cannot use division.

Example:

`nums = {1, 2, 3, 4}`

Output:

`{24, 12, 8, 6}`

16.2 How to Identify This Pattern

Each answer needs:

- product of elements to the left
- product of elements to the right

This is prefix/suffix thinking.

It is not prefix sum, but the same cumulative idea.

16.3 Brute Force Idea

For every index, multiply all other elements.

Time: $O(n^2)$

16.4 Optimized Intuition

Two passes:

First pass:

answer[i] = product of all elements before i

Second pass:

answer[i] *= product of all elements after i

16.5 C++ Code

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();

        vector<int> answer(n, 1);

        int prefixProduct = 1;

        for (int i = 0; i < n; i++) {
            answer[i] = prefixProduct;
            prefixProduct *= nums[i];
        }

        int suffixProduct = 1;

        for (int i = n - 1; i >= 0; i--) {
            answer[i] *= suffixProduct;
            suffixProduct *= nums[i];
        }

        return answer;
    }
};
```

16.6 Code Explanation

answer[i] = prefixProduct;

Stores product before current index.

prefixProduct *= nums[i];

Then include current for future indices.

Second pass:

answer[i] *= suffixProduct;

Multiplies by product after current index.

16.7 Dry Run

nums = {1, 2, 3, 4}

First pass:

0	1	1	1
1	1	1	2
2	2	2	6
3	6	6	24

Answer after first pass:

{1, 1, 2, 6}

Second pass:

3	1	6	4
2	4	8	12
1	12	12	24
0	24	24	24

Final:

{24, 12, 8, 6}

16.8 Complexity

Time: $O(n)$

Space: $O(1)$ extra, if output array does not count.

16.9 Edge Cases

- one zero
- two or more zeroes
- negative numbers
- one element
- product overflow depending on constraints

16.10 Common Mistakes

Wrong order:

```
prefixProduct *= nums[i];  
answer[i] = prefixProduct;
```

This includes current element, which should be excluded.

Correct:

```
answer[i] = prefixProduct;  
prefixProduct *= nums[i];
```

16.11 Similar Problems

- Trapping Rain Water
- Left and Right Sum Differences
- Prefix/suffix max problems

17. Full Problem Walkthrough 6 — LeetCode 560: Subarray Sum Equals K

17.1 Problem Summary

Given array `nums` and integer `k`, return the number of continuous subarrays whose sum equals `k`.

Example:

```
nums = {1, 1, 1}  
k = 2
```

Output:

```
2
```

17.2 How to Identify This Pattern

Problem asks:

- count subarrays
- exact sum `K`
- negative numbers may exist

Use prefix sum + hash map.

Sliding window is not safe with negatives.

17.3 Brute Force Idea

Try all subarrays.

Time: $O(n^2)$

17.4 Key Equation

Let:

```
prefix = sum from 0 to current index
```

For previous prefix `prev`:

```
subarray sum = prefix - prev
```

We want:

prefix - prev = k

So:

prev = prefix - k

17.5 C++ Code

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> prefixCount;

        prefixCount[0] = 1;

        int prefix = 0;
        int answer = 0;

        for (int x : nums) {
            prefix += x;

            int need = prefix - k;

            if (prefixCount.count(need)) {
                answer += prefixCount[need];
            }

            prefixCount[prefix]++;
        }

        return answer;
    }
};
```

17.6 Dry Run

nums = {1, 1, 1}

k = 2

Initial:

prefixCount = {0:1}

1	1	-1	0	0

x	prefix	need	added	answer
1	2	0	1	1
1	3	1	1	2

Return 2.

17.7 Dry Run With Negatives

nums = {1, -1, 1}
k = 1

1	1	0	1	1
-1	0	-1	0	1
1	1	0	2	3

Return 3.

17.8 Complexity

Time: $O(n)$ average

Space: $O(n)$

17.9 Edge Cases

- k = 0
- negative values
- all zeroes
- subarray starts at index 0
- repeated prefix sums

17.10 Common Mistakes

Mistake 1: Forgetting initialization

```
prefixCount[0] = 1;
```

Without it, subarrays starting at index 0 are missed.

Mistake 2: Using set

Need frequencies, not just existence.

Mistake 3: Updating map before checking

Correct order:

```
prefix += x;
answer += count[prefix - k];
count[prefix]++;
```

17.11 Similar Problems

- Binary Subarrays With Sum

- Count Number of Nice Subarrays
 - Contiguous Array
 - Subarray Sums Divisible by K
-

18. Full Problem Walkthrough 7 — LeetCode 525: Contiguous Array

18.1 Problem Summary

Given binary array `nums`, return maximum length of contiguous subarray with equal number of 0 and 1.

Example:

```
nums = {0, 1, 0}
```

Output:

```
2
```

18.2 How to Identify This Pattern

Problem asks:

- equal number of 0 and 1
- longest subarray

Transform:

```
0 -> -1  
1 -> +1
```

Then equal number of 0 and 1 means subarray sum is 0.

Use prefix sum + first index.

18.3 Key Insight

If the same prefix balance appears twice, the subarray between them has balance 0.
Meaning equal 0s and 1s.

18.4 C++ Code

```
class Solution {  
public:  
    int findMaxLength(vector<int>& nums) {  
        unordered_map<int, int> firstIndex;  
  
        firstIndex[0] = -1;  
  
        int prefix = 0;  
        int best = 0;  
  
        for (int i = 0; i < nums.size(); i++) {
```

```

    if (nums[i] == 0) {
        prefix += -1;
    } else {
        prefix += 1;
    }

    if (firstIndex.count(prefix)) {
        best = max(best, i - firstIndex[prefix]);
    } else {
        firstIndex[prefix] = i;
    }
}

return best;
}
};

```

18.5 Dry Run

nums = {0, 1, 0}

Transform conceptually:

{-1, +1, -1}

0	0	-1	store -1 at 0	0
1	1	0	seen 0 at -1, length 2	2
2	0	-1	seen -1 at 0, length 2	2

Return 2.

18.6 Complexity

Time: $O(n)$

Space: $O(n)$

18.7 Edge Cases

- all zeroes
- all ones
- whole array balanced
- no balanced subarray
- alternating values

18.8 Common Mistakes

Mistake 1: Mapping 0 to 0

Need 0 to cancel 1.

Correct:

0 -> -1

1 -> +1

Mistake 2: Overwriting first index

For longest length, keep earliest index.

18.9 Similar Problems

- Maximum Size Subarray Sum Equals K
- Longest Well-Performing Interval
- Subarray Sum Equals K

19. Full Problem Walkthrough 8 — LeetCode 974: Subarray Sums Divisible by K

19.1 Problem Summary

Given array `nums` and integer `k`, return number of non-empty subarrays whose sum is divisible by `k`.

Example:

`nums = {4, 5, 0, -2, -3, 1}`

`k = 5`

Output:

7

19.2 How to Identify This Pattern

Problem says:

- subarray sums
- divisible by `k`
- count

Use prefix modulo + hash map.

19.3 Key Insight

A subarray sum from previous prefix to current prefix is divisible by `k` if both prefix sums have the same remainder.

If:

`prefix1 % k == prefix2 % k`

Then:

`(prefix2 - prefix1) % k == 0`

19.4 C++ Code

```
class Solution {
public:
    int subarraysDivByK(vector<int>& nums, int k) {
        unordered_map<int, int> remainderCount;

        remainderCount[0] = 1;

        int prefix = 0;
        int answer = 0;

        for (int x : nums) {
            prefix += x;

            int rem = ((prefix % k) + k) % k;

            if (remainderCount.count(rem)) {
                answer += remainderCount[rem];
            }

            remainderCount[rem]++;
        }

        return answer;
    }
};
```

19.5 Dry Run

nums = {4, 5, 0, -2, -3, 1}
k = 5

4	4	4	0	0
5	9	4	1	1
0	9	4	2	3
-2	7	2	0	3
-3	4	4	3	6

x	prefix	rem	previous same rem	answer
1	5	0	1	7

Return 7.

19.6 Complexity

Time: $O(n)$

Space: $O(k)$, because only k possible remainders.

19.7 Edge Cases

- negative numbers
- $k = 1$
- prefix itself divisible by k
- repeated remainders
- all zeroes

19.8 Common Mistake

Not normalizing negative remainder.

Wrong:

```
int rem = prefix % k;
```

Correct:

```
int rem = ((prefix % k) + k) % k;
```

19.9 Similar Problems

- Continuous Subarray Sum
- Make Sum Divisible by P
- Subarray Sum Equals K

20. Full Problem Walkthrough 9 — LeetCode 523: Continuous Subarray Sum

20.1 Problem Summary

Given array `nums` and integer `k`, return `true` if there exists a continuous subarray of length at least 2 whose sum is a multiple of `k`.

Example:

```
nums = {23, 2, 4, 6, 7}
k = 6
```

Output:

```
true
```

Because:

{2, 4}

sum is 6.

20.2 How to Identify This Pattern

Problem says:

- subarray sum multiple of k
- length at least 2
- existence

Use prefix remainder + first index.

20.3 Key Insight

If the same remainder appears at two indices far enough apart, the subarray between them has sum divisible by k.

Need length at least 2.

20.4 C++ Code

```
class Solution {
public:
    bool checkSubarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> firstIndex;

        firstIndex[0] = -1;

        int prefix = 0;

        for (int i = 0; i < nums.size(); i++) {
            prefix += nums[i];

            int rem = prefix % k;

            if (firstIndex.count(rem)) {
                if (i - firstIndex[rem] >= 2) {
                    return true;
                }
            } else {
                firstIndex[rem] = i;
            }
        }

        return false;
    }
};
```

20.5 Dry Run

nums = {23, 2, 4, 6, 7}
k = 6

0	23	23	5	store 5 -> 0
1	2	25	1	store 1 -> 1
2	4	29	5	seen 5 at 0, length 2, true

Return true.

20.6 Complexity

Time: $O(n)$

Space: $O(k)$

20.7 Edge Cases

- length requirement at least 2
 - subarray starts at index 0
 - repeated remainder too close
 - zeros
 - k positive under modern constraints
-

20.8 Common Mistakes

Mistake 1: Forgetting length condition

Need:

$i - \text{firstIndex}[\text{rem}] \geq 2$

Mistake 2: Storing latest index

Store earliest index only, because earliest gives longest possible length.

20.9 Similar Problems

- Subarray Sums Divisible by K
 - Make Sum Divisible by P
 - Subarray Sum Equals K
-

21. Full Problem Walkthrough 10 — LeetCode 930: Binary Subarrays With Sum

21.1 Problem Summary

Given binary array nums and integer goal, return number of subarrays with sum exactly goal.

Example:

```
nums = {1, 0, 1, 0, 1}
```

```
goal = 2
```

Output:

```
4
```

21.2 How to Identify This Pattern

Problem asks:

- count subarrays
- exact sum
- binary array

Can be solved using either:

- prefix sum + hash map
- sliding window atMost trick

Here we use prefix sum.

21.3 C++ Code

```
class Solution {
public:
    int numSubarraysWithSum(vector<int>& nums, int goal) {
        unordered_map<int, int> count;

        count[0] = 1;

        int prefix = 0;
        int answer = 0;

        for (int x : nums) {
            prefix += x;

            if (count.count(prefix - goal)) {
                answer += count[prefix - goal];
            }

            count[prefix]++;
        }

        return answer;
    }
};
```

21.4 Dry Run Summary

For:

```
nums = {1, 0, 1, 0, 1}
goal = 2
```

Each time current prefix is prefix, we need previous prefix:

```
prefix - goal
```

All previous matching prefixes form valid subarrays ending here.

21.5 Complexity

Time: $O(n)$

Space: $O(n)$

21.6 Similar Problems

- Subarray Sum Equals K
 - Count Number of Nice Subarrays
 - Contiguous Array
-

22. Full Problem Walkthrough 11 — LeetCode 1248: Count Number of Nice Subarrays

22.1 Problem Summary

Given array `nums` and integer `k`, return number of subarrays with exactly `k` odd numbers.

Example:

```
nums = {1, 1, 2, 1, 1}
k = 3
```

Output:

```
2
```

22.2 How to Identify This Pattern

Odd numbers can be transformed:

```
odd -> 1
even -> 0
```

Then the problem becomes:

```
count subarrays with sum k
```

Use prefix sum count.

22.3 C++ Code

```
class Solution {
public:
    int numberOfSubarrays(vector<int>& nums, int k) {
        unordered_map<int, int> count;

        count[0] = 1;

        int prefix = 0;
        int answer = 0;

        for (int x : nums) {
            if (x % 2 == 1) {
                prefix++;
            }

            if (count.count(prefix - k)) {
                answer += count[prefix - k];
            }

            count[prefix]++;
        }

        return answer;
    }
};
```

22.4 Dry Run

```
nums = {1, 1, 2, 1, 1}
k = 3
```

Transformed:

```
{1, 1, 0, 1, 1}
```

Now count subarrays with sum 3.
Valid subarrays:

```
{1,1,2,1}
{1,2,1,1}
```

Answer 2.

22.5 Complexity

Time: $O(n)$
Space: $O(n)$

22.6 Alternative

Can also solve with sliding window:

```
atMost(k) - atMost(k - 1)
```

23. Full Problem Walkthrough 12 — LeetCode 304: Range Sum Query 2D — Immutable

23.1 Problem Summary

Given a matrix, answer rectangle sum queries.
Implement:

```
NumMatrix(vector<vector<int>>& matrix)  
int sumRegion(int row1, int col1, int row2, int col2)
```

23.2 How to Identify This Pattern

Problem asks:

- multiple matrix range sum queries
- immutable matrix
- rectangle sum

Use 2D prefix sum.

23.3 2D Prefix Formula

Build:

```
prefix[r + 1][c + 1]
```

as sum from (0,0) to (r,c).

Formula:

```
prefix[r + 1][c + 1] =  
    matrix[r][c]  
    + prefix[r][c + 1]  
    + prefix[r + 1][c]  
    - prefix[r][c];
```

Why subtract?

The top-left overlap was counted twice.

23.4 Query Formula

Sum from (row1, col1) to (row2, col2):

```
prefix[row2 + 1][col2 + 1]
- prefix[row1][col2 + 1]
- prefix[row2 + 1][col1]
+ prefix[row1][col1]
```

23.5 C++ Code

```
class NumMatrix {
private:
    vector<vector<int>>> prefix;

public:
    NumMatrix(vector<vector<int>>& matrix) {
        int rows = matrix.size();
        int cols = matrix[0].size();

        prefix.assign(rows + 1, vector<int>(cols + 1, 0));

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                prefix[r + 1][c + 1] =
                    matrix[r][c]
                    + prefix[r][c + 1]
                    + prefix[r + 1][c]
                    - prefix[r][c];
            }
        }
    }

    int sumRegion(int row1, int col1, int row2, int col2) {
        return prefix[row2 + 1][col2 + 1]
            - prefix[row1][col2 + 1]
            - prefix[row2 + 1][col1]
            + prefix[row1][col1];
    }
};
```

23.6 Complexity

Build time: $O(\text{rows} * \text{cols})$

Query time: $O(1)$

Space: $O(\text{rows} * \text{cols})$

23.7 Edge Cases

- single cell query
 - first row/column
 - entire matrix
 - negative values
 - one row or one column
-

23.8 Common Mistakes

Mistake 1: Forgetting overlap add-back

Need:

```
+ prefix[row1][col1]
```

Mistake 2: Not using extra row/column

Without extra row/column, boundary cases are harder.

23.9 Similar Problems

- Matrix Block Sum
 - Number of Submatrices That Sum to Target
 - Max Sum of Rectangle No Larger Than K
-

24. Full Problem Walkthrough 13 — LeetCode 1314: Matrix Block Sum

24.1 Problem Summary

Given matrix mat and integer k, return matrix answer where:

```
answer[i][j]
```

is sum of all elements within k distance from (i, j).

Example region:

```
rows from i-k to i+k
```

```
cols from j-k to j+k
```

clamped within matrix boundaries.

24.2 How to Identify This Pattern

Problem asks:

- many rectangle sums
- same matrix
- each cell needs a surrounding block sum

Use 2D prefix sum.

24.3 C++ Code

```
class Solution {  
public:
```

```

vector<vector<int>> matrixBlockSum(vector<vector<int>>& mat, int k) {
    int rows = mat.size();
    int cols = mat[0].size();

    vector<vector<int>> prefix(rows + 1, vector<int>(cols + 1, 0));

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            prefix[r + 1][c + 1] =
                mat[r][c]
                + prefix[r][c + 1]
                + prefix[r + 1][c]
                - prefix[r][c];
        }
    }

    vector<vector<int>> answer(rows, vector<int>(cols, 0));

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            int r1 = max(0, r - k);
            int c1 = max(0, c - k);
            int r2 = min(rows - 1, r + k);
            int c2 = min(cols - 1, c + k);

            answer[r][c] =
                prefix[r2 + 1][c2 + 1]
                - prefix[r1][c2 + 1]
                - prefix[r2 + 1][c1]
                + prefix[r1][c1];
        }
    }

    return answer;
}
};

```

24.4 Complexity

Build prefix: $O(\text{rows} * \text{cols})$

Answer all cells: $O(\text{rows} * \text{cols})$

Total: $O(\text{rows} * \text{cols})$

Space: $O(\text{rows} * \text{cols})$

24.5 Edge Cases

- $k = 0$
 - k larger than dimensions
 - single row/column
 - negative numbers
-

25. Full Problem Walkthrough 14 — LeetCode 325: Maximum Size Subarray Sum Equals K

This problem is LeetCode Premium, but it is very important.

25.1 Problem Summary

Given array `nums` and integer `k`, return maximum length of a subarray that sums to `k`.

Example:

```
nums = {1, -1, 5, -2, 3}
k = 3
```

Output:

```
4
```

Subarray:

```
{1, -1, 5, -2}
```

sum is 3.

25.2 How to Identify This Pattern

Problem asks:

- longest subarray
- exact sum `K`
- negative numbers may exist

Use prefix sum + first index.

25.3 Key Insight

If:

```
currentPrefix - previousPrefix = k
```

Then:

```
previousPrefix = currentPrefix - k
```

To maximize length, we want earliest index of `previousPrefix`.

25.4 C++ Code

```
int maxSubArrayLen(vector<int>& nums, int k) {
    unordered_map<int, int> firstIndex;

    firstIndex[0] = -1;

    int prefix = 0;
    int best = 0;

    for (int i = 0; i < nums.size(); i++) {
        prefix += nums[i];

        int need = prefix - k;

        if (firstIndex.count(need)) {
            best = max(best, i - firstIndex[need]);
        }

        if (!firstIndex.count(prefix)) {
            firstIndex[prefix] = i;
        }
    }

    return best;
}
```

25.5 Why Store First Index Only?

For longest length, earlier prefix gives longer subarray.
If you overwrite it, you may lose a longer answer.

25.6 Complexity

Time: $O(n)$

Space: $O(n)$

25.7 Similar Problems

- Contiguous Array
 - Subarray Sum Equals K
 - Longest Well-Performing Interval
-

26. Full Problem Walkthrough 15 —

LeetCode 1074: Number of Submatrices That Sum to Target

This is a harder prefix sum problem.

26.1 Problem Summary

Given matrix and target, return number of non-empty submatrices that sum to target.

Example:

```
matrix = {  
    {0, 1, 0},  
    {1, 1, 1},  
    {0, 1, 0}  
}  
target = 0
```

Output:

4

26.2 How to Identify This Pattern

Problem asks:

- count submatrices
- sum equals target

This is 2D version of Subarray Sum Equals K.

We can reduce 2D to multiple 1D problems.

26.3 Core Idea

Fix two row boundaries:

```
top  
bottom
```

Compress columns between those rows into a 1D array.

Then count subarrays in that compressed array with sum target.

Use Prefix Sum + Hash Map.

26.4 C++ Code

```
class Solution {  
public:  
    int subarraySum(vector<int>& nums, int target) {  
        unordered_map<int, int> count;  
  
        count[0] = 1;
```

```

int prefix = 0;
int answer = 0;

for (int x : nums) {
    prefix += x;

    if (count.count(prefix - target)) {
        answer += count[prefix - target];
    }

    count[prefix]++;
}

return answer;
}

int numSubmatrixSumTarget(vector<vector<int>>& matrix, int target) {
    int rows = matrix.size();
    int cols = matrix[0].size();

    int answer = 0;

    for (int top = 0; top < rows; top++) {
        vector<int> colSum(cols, 0);

        for (int bottom = top; bottom < rows; bottom++) {
            for (int c = 0; c < cols; c++) {
                colSum[c] += matrix[bottom][c];
            }

            answer += subarraySum(colSum, target);
        }
    }

    return answer;
}
};

```

26.5 Code Explanation

```
for (int top = 0; top < rows; top++)
```

Choose top row.

```
vector<int> colSum(cols, 0);
```

Compressed 1D array of column sums.

```
for (int bottom = top; bottom < rows; bottom++)
```

Expand bottom row.

```
colSum[c] += matrix[bottom][c];
```

Now colSum represents sums between row top and row bottom.
Then count subarrays of columns with target sum.

26.6 Complexity

Let:

R = rows

C = cols

Time:

$O(R^2 * C)$

Because there are $O(R^2)$ row pairs, and each compressed array is processed in $O(C)$.

Space:

$O(C)$

plus hash map.

If columns are fewer than rows, this is good.

If rows are much larger than columns, you can fix column boundaries instead.

26.7 Edge Cases

- negative numbers
- target zero
- single row
- single column
- all zero matrix
- large count of submatrices

26.8 Common Mistake

Trying to enumerate all four boundaries:

top, bottom, left, right

That becomes $O(R^2 * C^2)$, and checking sums may add more time.
Compression + 1D prefix count is much better.

27. Mini Problem 1 — LeetCode 1991: Find the Middle Index in Array

Pattern

Same as Pivot Index.

```
class Solution {
public:
    int findMiddleIndex(vector<int>& nums) {
        int total = 0;

        for (int x : nums) {
            total += x;
        }

        int leftSum = 0;

        for (int i = 0; i < nums.size(); i++) {
            int rightSum = total - leftSum - nums[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += nums[i];
        }

        return -1;
    }
};
```

28. Mini Problem 2 — LeetCode 2574: Left and Right Sum Differences

Pattern

Total sum + running left sum.

```
class Solution {
public:
    vector<int> leftRightDifference(vector<int>& nums) {
        int total = 0;

        for (int x : nums) {
```

```

        total += x;
    }

    int leftSum = 0;
    vector<int> answer(nums.size());

    for (int i = 0; i < nums.size(); i++) {
        int rightSum = total - leftSum - nums[i];

        answer[i] = abs(leftSum - rightSum);

        leftSum += nums[i];
    }

    return answer;
}
};

```

29. Mini Problem 3 — LeetCode 523 / 974 Difference

This is a concept mini-section.

974. Subarray Sums Divisible by K

Asks:

How many subarrays?

Use remainder count:

remainderCount[rem]++

523. Continuous Subarray Sum

Asks:

Does one exist with length at least 2?

Use first index:

firstIndex[rem]

Count vs existence with length changes which map value you store.

30. Mini Problem 4 — LeetCode 560 vs Sliding Window

For:

```
nums = {1, -1, 1}
k = 1
```

Valid subarrays:

```
{1}
{1, -1, 1}
{1}
```

Sliding window cannot reliably count these because sum can decrease when expanding.

Prefix sum works because it does not require monotonic movement.

31. Common Edge Cases for Prefix Sum

31.1 Empty Array

For prefix array:

```
vector<int> prefix(nums.size() + 1, 0);
```

works.

But avoid accessing:

```
nums[0]
```

if array may be empty.

31.2 Range Starts at 0

Exclusive prefix handles this naturally:

```
prefix[r + 1] - prefix[0]
```

31.3 Negative Numbers

Prefix sum works with negative numbers.

Sliding window may fail.

31.4 $k = 0$

For exact sum K , prefix sum works.

For modulo problems, modulo by zero is invalid.

Check constraints.

31.5 Large Sum

Use:

```
long long prefix = 0;
unordered_map<long long, int> count;
```

if values can overflow int.

31.6 Negative Modulo

Use:

```
int rem = ((prefix % k) + k) % k;
```

31.7 Subarray Length Requirement

For length at least 2:

```
i - firstIndex[rem] >= 2
```

31.8 Duplicate Prefix Sums

Duplicate prefix sums mean zero-sum subarray between them.

For counting, store frequency.

For longest length, store first index.

32. Common Mistakes and Wrong Code

Mistake 1 — Off-by-One in Range Sum

Wrong for exclusive prefix:

```
prefix[right] - prefix[left]
```

Correct:

```
prefix[right + 1] - prefix[left]
```

Mistake 2 — Forgetting Initial Prefix

For count problems:

```
count[0] = 1;
```

For first index problems:

```
firstIndex[0] = -1;
```

Mistake 3 — Using Set Instead of Map

For counting subarrays, need frequency.

Wrong:

```
unordered_set<int> seen;
```

Correct:

```
unordered_map<int, int> count;
```

Mistake 4 — Overwriting First Index

Wrong:

```
firstIndex[prefix] = i;
```

Correct:

```
if (!firstIndex.count(prefix)) {  
    firstIndex[prefix] = i;  
}
```

Mistake 5 — Not Normalizing Negative Remainders

Wrong:

```
int rem = prefix % k;
```

Correct:

```
int rem = ((prefix % k) + k) % k;
```

Mistake 6 — Using Sliding Window for Negative Exact Sum

If negative values exist and exact sum is needed, use prefix sum + hash map.

33. Interview Explanation Templates

33.1 Range Sum Query

I precompute a prefix sum array where $\text{prefix}[i + 1]$ stores the sum of elements before index $i + 1$. Then the sum from left to right is $\text{prefix}[\text{right} + 1] - \text{prefix}[\text{left}]$. This gives $O(1)$ query time after $O(n)$ preprocessing.

33.2 Subarray Sum Equals K

I maintain a running prefix sum. A subarray ending at the current index has sum k if there was a previous prefix sum equal to $\text{currentPrefix} - k$. I store counts of prefix sums in a hash map and add the matching count to the answer.

33.3 Longest Subarray Sum K

I store the earliest index where each prefix sum appears. If $\text{currentPrefix} - k$ appeared before, the subarray after that index to the current index sums to k . Earliest index gives the longest length.

33.4 Divisible by K

Two prefix sums with the same remainder modulo k form a subarray whose sum is divisible by k . So I count how many times each remainder has appeared.

33.5 Pivot Index

I compute the total sum first. As I scan, I maintain leftSum . The right sum is $\text{total} - \text{leftSum} - \text{nums}[i]$. If left and right are equal, I return the index.

34. Complexity Patterns

Running sum	$O(n)$	$O(n)$ output
Range sum query	$O(n)$ build, $O(1)$ query	$O(n)$
Pivot index	$O(n)$	$O(1)$
Product except self	$O(n)$	$O(1)$ extra
Subarray sum equals K	$O(n)$ average	$O(n)$
Longest subarray sum K	$O(n)$ average	$O(n)$
Subarray divisible by K	$O(n)$ average	$O(k)$
Contiguous Array	$O(n)$ average	$O(n)$
2D prefix sum	$O(R \cdot C)$ build, $O(1)$ query	$O(R \cdot C)$
Number of submatrices target	$O(R^2 \cdot C)$	$O(C)$

35. Time-Boxed Practice Guidance

Easy Prefix Problems

Target:

5–12 minutes

Examples:

- Running Sum
- Highest Altitude
- Pivot Index
- Left and Right Sum Differences

Medium Prefix + Hash Map Problems

Target:

20–35 minutes

Examples:

- Subarray Sum Equals K
- Contiguous Array
- Subarray Sums Divisible by K
- Continuous Subarray Sum

Harder 2D Prefix Problems

Target:

35–60 minutes

Examples:

- Range Sum Query 2D
 - Matrix Block Sum
 - Number of Submatrices That Sum to Target
-

36. Expanded Practice Set

36.1 Must-Do Easy

- 1.1. Running Sum of 1D Array
 - 2.2. Find the Highest Altitude
 - 3.3. Find Pivot Index
 - 4.4. Find the Middle Index in Array
 - 5.5. Left and Right Sum Differences
 - 6.6. Range Sum Query — Immutable
-

36.2 Must-Do Medium

- 7.7. Product of Array Except Self
 - 8.8. Subarray Sum Equals K
 - 9.9. Contiguous Array
 - 10.10. Subarray Sums Divisible by K
 - 11.11. Continuous Subarray Sum
 - 12.12. Binary Subarrays With Sum
 - 13.13. Count Number of Nice Subarrays
 - 14.14. Range Sum Query 2D — Immutable
 - 15.15. Matrix Block Sum
 - 16.16. Number of Sub-arrays With Odd Sum
-

36.3 Strong Interview Practice

- 17.17. Maximum Size Subarray Sum Equals K
 - 18.18. Number of Submatrices That Sum to Target
 - 19.19. Make Sum Divisible by P
 - 20.20. Longest Well-Performing Interval
 - 21.21. Path Sum III, tree prefix sum
 - 22.22. Subarray Sum Equals K
 - 23.23. Subarray Sums Divisible by K
 - 24.24. Continuous Subarray Sum
-

36.4 Hard / Stretch

- 25.25. Number of Submatrices That Sum to Target
 - 26.26. Max Sum of Rectangle No Larger Than K
 - 27.27. Count of Range Sum
 - 28.28. Shortest Subarray with Sum at Least K, prefix + monotonic deque
 - 29.29. Sum of Total Strength of Wizards, advanced prefix + monotonic stack
-

37. Suggested Study Order

Step 1 — Basic Prefix

12. Running Sum of 1D Array
13. Find the Highest Altitude
14. Find Pivot Index
15. Left and Right Sum Differences

Goal:

Understand cumulative sums and left/right sum logic.

Step 2 — Range Queries

16. Range Sum Query — Immutable
17. Range Sum Query 2D — Immutable
18. Matrix Block Sum

Goal:

Master exclusive prefix arrays and 2D inclusion-exclusion.

Step 3 — Prefix/Suffix

19. Product of Array Except Self
20. Left and Right Sum Differences

Goal:

Understand left-side and right-side cumulative information.

Step 4 — Prefix + Hash Map Count

21. Subarray Sum Equals K
22. Binary Subarrays With Sum
23. Count Number of Nice Subarrays

Goal:

Learn how to count subarrays using previous prefix frequencies.

Step 5 — Prefix + First Index

24. Contiguous Array
25. Maximum Size Subarray Sum Equals K
26. Longest Well-Performing Interval

Goal:

Learn longest length using earliest prefix index.

Step 6 — Prefix Modulo

27. Subarray Sums Divisible by K
28. Continuous Subarray Sum
29. Make Sum Divisible by P

Goal:

Understand same remainder logic.

Step 7 — Advanced 2D Prefix

30. Matrix Block Sum
31. Number of Submatrices That Sum to Target
32. Max Sum of Rectangle No Larger Than K

Goal:

38. Pattern Decision Table

Running cumulative total	running prefix	prefix += x
Many range sums	prefix array	prefix[r+1] - prefix[l]
Left sum equals right sum	total + leftSum	Pivot Index
Product except self	prefix/suffix product	two passes
Count subarrays sum K	prefix count map	count[prefix-k]
Longest subarray sum K	first index map	earliest prefix
Equal 0s and 1s	transform 0 to -1	prefix first index
Divisible by K	remainder count	same remainder
Multiple of K with length	remainder first index	check length
Matrix rectangle sum	2D prefix	inclusion-exclusion
Count submatrices target	row compression + prefix map	2D to 1D

39. Final Cheat Sheet

Exclusive Prefix Array

```
vector<int> prefix(nums.size() + 1, 0);
```

```
for (int i = 0; i < nums.size(); i++) {  
    prefix[i + 1] = prefix[i] + nums[i];  
}
```

```
int rangeSum = prefix[r + 1] - prefix[l];
```

Pivot Index

```

int total = 0;

for (int x : nums) {
    total += x;
}

int leftSum = 0;

for (int i = 0; i < nums.size(); i++) {
    int rightSum = total - leftSum - nums[i];

    if (leftSum == rightSum) {
        return i;
    }

    leftSum += nums[i];
}

return -1;

```

Subarray Sum Equals K

```

unordered_map<int, int> count;
count[0] = 1;

int prefix = 0;
int answer = 0;

for (int x : nums) {
    prefix += x;

    if (count.count(prefix - k)) {
        answer += count[prefix - k];
    }

    count[prefix]++;
}

return answer;

```

Longest Subarray Sum K

```

unordered_map<int, int> firstIndex;
firstIndex[0] = -1;

int prefix = 0;
int best = 0;

for (int i = 0; i < nums.size(); i++) {
    prefix += nums[i];

    if (firstIndex.count(prefix - k)) {
        best = max(best, i - firstIndex[prefix - k]);
    }

    if (!firstIndex.count(prefix)) {
        firstIndex[prefix] = i;
    }
}

return best;

```

Divisible by K Count

```

unordered_map<int, int> count;
count[0] = 1;

int prefix = 0;
int answer = 0;

for (int x : nums) {
    prefix += x;

    int rem = ((prefix % k) + k) % k;

    answer += count[rem];

    count[rem]++;
}

return answer;

```

Product Except Self

```

vector<int> answer(nums.size(), 1);

int prefixProduct = 1;

for (int i = 0; i < nums.size(); i++) {
    answer[i] = prefixProduct;
    prefixProduct *= nums[i];
}

int suffixProduct = 1;

for (int i = nums.size() - 1; i >= 0; i--) {
    answer[i] *= suffixProduct;
    suffixProduct *= nums[i];
}

return answer;

```

2D Prefix Query

```

int sum =
    prefix[r2 + 1][c2 + 1]
    - prefix[r1][c2 + 1]
    - prefix[r2 + 1][c1]
    + prefix[r1][c1];

```

40. What to Master Before Chapter 6

Before moving to Difference Array, make sure you can:

- Build exclusive prefix arrays
- Query range sums with $\text{prefix}[r + 1] - \text{prefix}[l]$
- Avoid off-by-one errors
- Use running prefix without storing array
- Solve Pivot Index
- Understand prefix/suffix products
- Derive $\text{previousPrefix} = \text{currentPrefix} - k$
- Use hash map counts for subarray count
- Use first-index map for longest subarray
- Normalize negative modulo
- Use prefix modulo for divisibility
- Build 2D prefix sums
- Explain when prefix sum beats sliding window

41. Core Takeaway

Prefix Sum is about turning repeated range calculations into subtraction.

The core formulas are:

$\text{prefix}[i + 1] = \text{prefix}[i] + \text{nums}[i]$

$\text{sum}(l, r) = \text{prefix}[r + 1] - \text{prefix}[l]$

For subarray sum problems:

$\text{currentPrefix} - \text{previousPrefix} = \text{target}$

So:

$\text{previousPrefix} = \text{currentPrefix} - \text{target}$

That equation is the heart of many prefix sum + hash map problems.

If you master prefix sum, you unlock a huge class of interview problems involving subarrays, ranges, balances, divisibility, and matrix sums.
